

Clarity — B2B Invoice Discounting

সম্পূর্ণ প্রজেক্ট গাইড

CSE471 · System Analysis and Design

অপূর্ব রায় (Apurba Roy) · ID 23101012 · Section 09 · Group 4

ফোল্ডার স্ট্রাকচার · ফিচার · টেস্টিং · লাইন-বাই-লাইন কোড · API · ভাইভা প্রস্তুতি

১. প্রজেক্ট পরিচিতি — এক নজরে

বাংলাদেশের ছোট ও মাঝারি সাপ্লায়াররা বড় কোম্পানিকে মাল সরবরাহ করার পর ৯০ থেকে ১২০ দিন টাকার জন্য অপেক্ষা করে। এই সময়টায় ব্যবসা চালাতে তারা মহাজনের কাছ থেকে বছরে ৩১% সুদে টাকা ধার করে।

Clarity একটি ওয়েব প্ল্যাটফর্ম যা এই সমস্যার সমাধান করে। সাপ্লায়ার তার confirmed invoice আপলোড করে, আর ৭২ ঘণ্টার মধ্যে invoice-এর ৯৭% পর্যন্ত টাকা পেয়ে যায় — অল্প একটু discount দিয়ে।

ব্যবহারকারী	কী করে
Supplier (সাপ্লায়ার)	Invoice আপলোড করে, আগেভাগে টাকা পায়
Buyer (ক্রেতা)	Invoice নিশ্চিত করে অথবা dispute করে
Funder (ব্যাংক/NBFI)	টাকা দেয়, রিটার্ন পায়
Admin	KYB যাচাই, dispute-এর মীমাংসা

আমার দায়িত্ব — দুটি ফিচার

Module	ফিচারের নাম
Module 1	Invoice Upload with OCR Parsing & Cloud Storage
Module 2	Dispute Filing & Invoice Freeze

আমার External API: **Cloudinary** (invoice-এর ডকুমেন্ট নিরাপদে জমা রাখার জন্য)।

২. ফোল্ডার স্ট্রাকচার ও আর্কিটেকচার

২.১ আর্কিটেকচার প্যাটার্ন

এই প্রজেক্ট **Feature-First + Layered Architecture** অনুসরণ করে। মানে —

- **Feature-First:** ক্লায়েন্টে প্রত্যেক গ্রুপ মেম্বারের কোড আলাদা ফোল্ডারে থাকে (`apurba/`, `mihir/`, `digonto/`, `ameet/`)। ফলে চারজন একসাথে কাজ করলেও কারো ফাইল কারো সাথে সংঘর্ষ করে না।
- **Layered:** সার্ভারে কাজ তিন স্তরে ভাগ করা — *routes* (কোন URL কী করবে), *db* (ডেটাবেস কানেকশন), *sql* (টেবিলের গঠন)।

পুরো সিস্টেমে তিনটি স্তর:

Browser (React + Vite) localhost:5173	Express Server localhost:5001	PostgreSQL Supabase (Tokyo)
InvoiceUpload.jsx	<code>/api/invoices</code>	<code>invoices</code>
DisputeCentre.jsx	<code>/api/disputes</code>	<code>disputes</code>
	←JSON→	←rows→ <code>dispute_evidence</code>

গুরুত্বপূর্ণ প্রশ্ন

ব্রাউজার সরাসরি ডেটাবেসে কেন কথা বলে না? তিনটি কারণ — (১) *Security*: ডেটাবেসের পাসওয়ার্ড তাহলে প্রত্যেক ব্রাউজারে চলে যেত। (২) *Validation*: ব্রাউজারের পাঠানো ডেটা যে কেউ বদলে দিতে পারে, তাই সার্ভারেই আসল যাচাই হতে হবে। (৩) *Protocol*: PostgreSQL যে ভাষায় কথা বলে ব্রাউজার সেটা বোঝেই না।

২.২ ফোল্ডার ম্যাপ

```
Clarity-supply-chain-finance/
├── mergedCode_M1/           ← আসল প্রজেক্ট (চারজনের কোড একসাথে)
│   ├── client/             ← Frontend (React + Vite)
│   │   ├── index.html      ← ব্রাউজার যে ফাইলটা প্রথমে খোলে
│   │   ├── vite.config.ts  ← Vite-এর সেটিংস + proxy
│   │   └── src/
│   │       ├── main.jsx     ← React অ্যাপ চালু করার ফাইল
│   │       ├── App.jsx      ← কোন URL-এ কোন পেজ দেখাবে (routing)
│   │       ├── components/  ← সবার জন্য কমন – Sidebar, Header
│   │       ├── apurba/      ← ★ আমার পেজগুলো
│   │       ├── mihir/       ← Mihir-এর পেজ
│   │       ├── digonto/     ← Digonta-র পেজ
│   │       └── ameer/       ← Ameer-এর পেজ
│   └── server/             ← Backend (Node.js + Express)
│       ├── index.js        ← সার্ভার চালু হয় এখান থেকে
│       ├── db.js           ← PostgreSQL কানেকশন
│       ├── .env            ← গোপন পাসওয়ার্ড ও key (git-এ যায় না)
│       ├── routes/         ← প্রত্যেক ফিচারের API আলাদা ফাইলে
│       │   ├── invoiceRoutes.js ← ★ আমার Feature 1
│       │   ├── disputeRoutes.js ← ★ আমার Feature 2
│       │   ├── payoutRoutes.js  ← আমার সাপোর্টিং পেজ
│       │   ├── pipelineRoutes.js ← Mihir
│       │   ├── cashflowRoutes.js ← Ameer
│       │   └── confirmationRoutes.js ← Digonta
│       └── sql/
│           ├── schema.sql    ← টেবিল তৈরির SQL
│           └── seed.sql      ← নমুনা ডেটা
└── sample-invoices/        ← টেস্ট করার জন্য নমুনা invoice ফাইল
```

আমার নিজের ফাইলগুলো

ফাইল	কাজ
client/src/apurba/InvoiceUpload.jsx	Feature 1-এর পুরো UI — ফাইল ড্র্যাগ, OCR, ফর্ম
client/src/apurba/DisputeCentre.jsx	Feature 2-এর পুরো UI — dispute করা ও মীমাংসা
client/src/apurba/MyInvoices.jsx	সব invoice-এর তালিকা
client/src/apurba/PayoutHistory.jsx	টাকা পাওয়ার হিসাব
client/src/apurba/styles.css	আমার পেজগুলোর ডিজাইন
server/routes/invoiceRoutes.js	Feature 1-এর ৭টি API
server/routes/disputeRoutes.js	Feature 2-এর ৭টি API

৩. ফিচার বিশ্লেষণ

ফিচার ১ — Invoice Upload with OCR Parsing & Cloud Storage

কী করে

সাপ্লায়ার তার invoice-এর PDF বা ছবি আপলোড করে। সিস্টেম নিজে থেকেই সেই কাগজ পড়ে চারটি তথ্য বের করে আনে — **ক্রেতার নাম**, **invoice নম্বর**, **মোট টাকা**, **আর শেষ তারিখ** — এবং ফর্মটা আগে থেকেই পূরণ করে দেয়। সাপ্লায়ার শুধু দেখে নেয় ঠিক আছে কিনা, তারপর submit করে।

কেন দরকার

- **ভুল কমানো:** লক্ষ টাকার কাগজে হাতে টাইপ করতে গিয়ে একটা শূন্য বেশি-কম হলে বিরাট ক্ষতি।
- **সময় বাঁচানো:** যে সাপ্লায়ারের দিনে ৫০টা invoice, তার জন্য হাতে টাইপ করা অসম্ভব।
- **Duplicate ঠেকানো:** একই invoice দুইবার জমা দিয়ে দুইবার টাকা তোলা — এটাই এই ব্যবসার সবচেয়ে বড় জালিয়াতি। সিস্টেম আগেই ধরে ফেলে।
- **প্রমাণ সংরক্ষণ:** আসল কাগজটা Cloudinary-তে জমা থাকে, পরে ঝামেলা হলে দেখানো যায়।

User Flow — ধাপে ধাপে

1. সাপ্লায়ার **Upload Invoice** পেজে যায়
2. invoice-এর ছবি বা PDF ড্র্যাগ করে ছেড়ে দেয়
3. PDF হলে ভেতরের লেখা সরাসরি পড়া হয়; ছবি হলে **OCR** চলে
4. চারটি ঘর নিজে থেকে পূরণ হয়ে যায়
5. invoice নম্বরটা আগে জমা পড়েছে কিনা সিস্টেম যাচাই করে — পড়ে থাকলে লাল সতর্কবার্তা
6. সাপ্লায়ার ভুল থাকলে ঠিক করে
7. **Confirm and submit** চাপে
8. ফাইলটা Cloudinary-তে জমা হয়, তারপর invoice ডেটাবেসে সেভ হয়

ফিচার ২ — Dispute Filing & Invoice Freeze

কী করে

ক্রেতা ৫০০ পিসের invoice পেল কিন্তু আসলে এসেছে ৩৮০ পিস। সে **dispute** করে — কারণ লেখে, প্রমাণের কাগজ যোগ করে। সঙ্গে সঙ্গে invoice-টা **freeze** হয়ে যায়, অর্থাৎ কোনো funder আর সেটাতে

টাকা লাগাতে পারে না। Admin দুই পক্ষের কথা শুনে সিদ্ধান্ত দেয় — *release* (আবার চালু) বা *void* (একেবারে বাতিল)।

কেন দরকার

- **Funder-কে বাঁচানো:** যে invoice নিয়ে ঝগড়া চলছে, তাতে টাকা লাগালে funder ফেরত নাও পেতে পারে।
- **ক্রেতার সুরক্ষা:** মাল পায়নি অথচ টাকা দিতে বাধ্য — এটা অন্যায়।
- **প্রমাণের ইতিহাস:** কে কখন কী করেছে সব লেখা থাকে, পরে কেউ অস্বীকার করতে পারে না।

User Flow — ধাপে ধাপে

1. ক্রেতা **Dispute Centre** পেজে যায়
2. কোন invoice নিয়ে সমস্যা সেটা বেছে নেয়
3. কারণ লেখে, প্রয়োজনে বিস্তারিত নোট দেয়
4. **File and freeze** চাপে → invoice সঙ্গে সঙ্গে freeze
5. প্রমাণের কাগজ (ডেলিভারি চালান, ছবি) আপলোড করে — Cloudinary-তে জমা হয়
6. Admin dispute দেখে **Release** বা **Void** চাপে
7. Release হলে invoice আবার marketplace-এ ফিরে যায়

খেয়াল করুন

Dispute করলে invoice-এর `status` কলামে কিছু লেখা হয় না। আলাদা `frozen_at` কলামে সময় বসানো হয়। কারণ `status` কলামটা Mihir-এর *state machine* ফিচারের সম্পত্তি — নিয়ম হলো ওই সার্ভিস ছাড়া কেউ ওটা লিখবে না। এটা ভাইভাতে বললে দেখা যায় যে আমি দলের নিয়ম মেনে কোড করেছি।

৪. টেস্টিং গাইড

৪.১ সার্ভার চালু করা

```
# Terminal 1 – Backend
cd mergedCode_M1/server
npm install
npm run setup      # টেবিল তৈরি + নমুনা ডেটা
npm start          # চলবে port 5001-এ

# Terminal 2 – Frontend
cd mergedCode_M1/client
npm install
npm run dev         # চলবে port 5173-এ
```

ব্রাউজারে `http://localhost:5173` খুলুন।

৪.২ ফিচার ১ টেস্ট — ধাপে ধাপে

ধাপ	কী করবেন	কী দেখলে বুঝবেন ঠিক আছে
১	<code>sample-invoices/1-standard.pdf</code> ড্র্যাগ করুন	সঙ্গে সঙ্গে চারটি ঘর পূরণ; লেখা থাকবে “read straight from the PDF” (OCR চলেনি)
২	তারিখ দেখুন	<code>10/11/2026</code> → 10 Nov 2026 (দিন আগে, মাস পরে — বাংলাদেশি নিয়ম)
৩	<code>4-photo-clean.png</code> ড্র্যাগ করুন	progress bar চলবে, “photograph, read by OCR” লেখা আসবে
৪	<code>5-photo-skewed.png</code> ড্র্যাগ করুন	কিছু ঘর ভুল আসবে — এটাই প্রমাণ করে কেন মানুষের যাচাই দরকার
৫	একই invoice নম্বর দুইবার submit করুন	লাল সতর্কবার্তা, submit বোতাম নিষ্ক্রিয়
৬	Submit করুন	সবুজ বার্তা: “...submitted and the document was stored ”

৪.৩ ফিচার ২ টেস্ট — ধাপে ধাপে

ধাপ	কী করবেন	কী দেখলে বুঝবেন ঠিক আছে
১	Dispute Centre → invoice বেছে কারণ লিখে File and freeze	সবুজ বার্তা “Dispute #1 filed. The invoice is now frozen.”
২	একই invoice-এ আবার dispute করুন	লাল বার্তা: “already frozen by an open dispute” — এটাই lock-এর প্রমাণ
৩	একটা ছবি বেছে Attach document	Documents (1)-এ ক্লিকযোগ্য লিংক আসবে
৪	লিংকে ক্লিক করুন	Cloudinary থেকে ছবি খুলবে
৫	Release invoice চাপুন	চিপ লাল “Open” থেকে সবুজ “Released” হবে; বোতামগুলো চলে যাবে
৬	History দেখুন	চারটি লাইন — filed, evidence added, released

৪.৪ Postman দিয়ে API টেস্ট

Postman খুলে `postman/Clarity-Apurba-23101012.postman_collection.json` ফাইলটা Import করুন। ১৩টি request পাওয়া যাবে। প্রত্যেকটাতে **Send** চাপুন।

HTTP Status	মানে	কোথায় দেখবেন
200 OK	চেয়েছেন, পেয়েছেন	সব GET
201 Created	নতুন কিছু তৈরি হয়েছে	invoice তৈরি, dispute ফাইল
400 Bad Request	আপনি ভুল পাঠিয়েছেন	ঘর খালি, টাকা সংখ্যা নয়
404 Not Found	এমন কিছু নেই	ভুল id
409 Conflict	অনুরোধ ঠিক, কিন্তু এখনকার অবস্থায় করা যাবে না	duplicate invoice, আগেই frozen
500 Server Error	আমাদের ভুল, আপনার নয়	ডেটাবেস বন্ধ

৪.৫ ডেটাবেসে যাচাই

Supabase-এ লগইন করে **Table Editor** খুলুন। দেখুন —

- `invoices` টেবিলে নতুন সারি এসেছে, `file_url` কলামে Cloudinary-র লিংক আছে
- `disputes` টেবিলে dispute-টা আছে, `status` = Open বা Released
- `dispute_events` টেবিলে ইতিহাসের প্রতিটি লাইন আছে

Cloudinary-তে লগইন করে **Assets** → **clarity/invoices** ফোল্ডারে গেলে আপলোড করা ফাইলটা দেখা যাবে।

৫. কোড — লাইন বাই লাইন ব্যাখ্যা

৫.১ server/db.js — ডেটাবেস কানেকশন

```
const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false },
});

module.exports = pool;
```

লাইন	ব্যাখ্যা
<code>require('pg')</code>	<code>pg</code> হলো Node.js-এর PostgreSQL ড্রাইভার — এটাই ডেটাবেসের সাথে কথা বলে
<code>new Pool(...)</code>	Pool মানে কয়েকটা কানেকশন খুলে রাখা। প্রতিবার নতুন কানেকশন খুললে ২০০-৩০০ মিলিসেকেন্ড নষ্ট হতো
<code>process.env.DATABASE_URL</code>	পাসওয়ার্ড কোডে লেখা নেই — <code>.env</code> ফাইল থেকে আসে, আর সেই ফাইল git-এ যায় না
<code>ssl</code>	Supabase এনক্রিপশন ছাড়া কানেকশন নেয় না
<code>module.exports = pool</code>	Node মডিউল একবারই লোড করে, তাই পুরো অ্যাপে একটাই pool থাকে

৫.২ server/index.js — সার্ভার চালু

```
const app = express();
const PORT = process.env.PORT || 5001;

app.use(cors());
app.use(express.json({ limit: '10mb' }));

app.use('/api/invoices', invoiceRoutes);
app.use('/api/disputes', disputeRoutes);
app.use('/api/payouts', payoutRoutes);
```

- `cors()` — ব্রাউজার নিয়ম করে রেখেছে যে ৫১৭৩ পোর্টের পেজ ৫০০১ পোর্টের উত্তর পড়তে পারবে না (*same-origin policy*)। `cors()` সেই অনুমতি দেয়।
- `express.json({ limit: '10mb' })` — POST-এর JSON body পড়ে `req.body`-তে রাখে। ১০ মেগাবাইট কারণ invoice-এর ছবি base64 করলে আসল ফাইলের চেয়ে ৩৩% বড় হয়, ডিফল্ট ১০০

কিলোবাইটে আটকে যেত।

- `app.use('/api/invoices', invoiceRoutes)` — এর মানে `invoiceRoutes` -এর ভেতরে লেখা `router.get('/')` আসলে `/api/invoices` হয়ে যায়। এভাবেই চারজনের API আলাদা থাকে।

৫.৩ `routes/invoiceRoutes.js` — ফিচার ১

(ক) Validation ফাংশন

```
function findProblems({ buyer_name, invoice_number, invoice_amount, due_date }) {  
  const problems = [];  
  
  if (!buyer_name) problems.push('buyer name is missing');  
  if (!invoice_number) problems.push('invoice number is missing');  
  
  if (!invoice_amount) {  
    problems.push('amount is missing');  
  } else if (isNaN(Number(invoice_amount)) || Number(invoice_amount) <= 0) {  
    problems.push('amount must be a number greater than zero');  
  }  
  
  if (!due_date) {  
    problems.push('due date is missing');  
  } else if (!/^\\d{4}-\\d{2}-\\d{2}$/.test(due_date)) {  
    problems.push('due date must be written as YYYY-MM-DD');  
  }  
  
  return problems;  
}
```

- ভুলগুলো একটি **array**-তে জমা হয়, একটা পেয়েই থামে না — তাই ব্যবহারকারী একবারেই সব ভুল জানতে পারে।
- `isNaN(Number(x))` — টাকার ঘরে "abc" লিখলে ধরা পড়ে।
- `/^\\d{4}-\\d{2}-\\d{2}$` একটি **regular expression**। মানে: শুরুতে ৪টা সংখ্যা, তারপর হাইফেন, ২টা সংখ্যা, হাইফেন, ২টা সংখ্যা, তারপর শেষ। `^` = শুরু, `$` = শেষ, `\\d` = যেকোনো সংখ্যা।

(খ) Duplicate যাচাই — API 1

```
router.post('/check-duplicate', async (req, res) => {
  const { invoice_number, supplier_id } = req.body;

  if (!invoice_number) {
    return res.status(400).json({ message: 'invoice_number is required' });
  }

  const found = await pool.query(
    `SELECT id, invoice_number, buyer_name, invoice_amount, submitted_date
    FROM invoices
    WHERE invoice_number = $1 AND supplier_id = $2`,
    [invoice_number, String(supplier_id || 1)]
  );

  res.json({
    invoice_number: invoice_number,
    duplicate: found.rowCount > 0,
    existing: found.rows[0] || null,
  });
});
```

অংশ	ব্যাখ্যা
<code>async / await</code>	ডেটাবেসের উত্তর আসতে সময় লাগে। <code>await</code> এই একটা অনুরোধকে অপেক্ষা করায়, কিন্তু সার্ভার এই ফাঁকে অন্য মানুষের অনুরোধ সামলাতে থাকে
<code>\$1, \$2</code>	সবচেয়ে গুরুত্বপূর্ণ। এগুলো <i>placeholder</i> । মানটা আলাদাভাবে array-তে যায়। PostgreSQL আগে query বোঝে, পরে মান বসায় — তাই মানটা কখনোই কমান্ড হিসেবে চলতে পারে না। এটাই SQL Injection ঠেকায়
<code>rowCount > 0</code>	একটাও সারি পাওয়া গেলে duplicate
<code>return</code>	<code>res.status(400)</code> -এর আগে <code>return</code> না দিলে কোড চলতেই থাকবে এবং দুইবার উত্তর পাঠানোর চেষ্টা করে crash করবে

(গ) Invoice সেভ করা — API 2

```
const clash = await pool.query(
  'SELECT id FROM invoices WHERE invoice_number = $1 AND supplier_id = $2',
  [invoice_number, String(supplier_id || 1)]
);
if (clash.rowCount > 0) {
  return res.status(409).json({ message: 'That invoice number has already been submitted' });
}
```

ভাইভাতে জিজ্ঞেস করতে পারে

“উপরে তো একবার **duplicate** চেক করেছেন, এখানে আবার কেন?” — কারণ ব্রাউজারের চেকটা শুধু ব্যবহারকারীর সুবিধার জন্য। কেউ Postman দিয়ে সরাসরি সার্ভারে অনুরোধ পাঠালে ওই চেক এড়িয়ে যাবে। সার্ভারই একমাত্র জায়গা যেখানে নিয়ম সত্যিই প্রয়োগ করা যায়।

(ঘ) Cloudinary-তে ফাইল আপলোড — API 0

```
const cloudinary = require('cloudinary').v2;

router.post('/upload-file', async (req, res) => {
  const { file, file_name } = req.body;

  if (!file) return res.status(400).json({ message: 'file is required' });
  if (!file.startsWith('data:'))
    return res.status(400).json({ message: 'file must be a data URI' });

  const uploaded = await cloudinary.uploader.upload(file, {
    folder: 'clarity/invoices',
    resource_type: 'auto',
  });

  res.status(201).json({ file_url: uploaded.secure_url, ... });
});
```

- SDK নিজেই `.env` থেকে `CLOUDINARY_URL` পড়ে নেয় — তাই cloud name, key বা secret কোডের কোথাও লেখা নেই।
- `resource_type: 'auto'` — ছবি নাকি PDF, Cloudinary নিজেই বুঝে নেয়।
- `secure_url` — `https` দিয়ে শুরু হওয়া লিংক, এটাই ডেটাবেসে জমা হয়।

৫.৪ routes/disputeRoutes.js — ফিচার ২ (সবচেয়ে গুরুত্বপূর্ণ কোড)

```
router.post('/', async (req, res) => {
  const { invoice_id, filed_by, reason, notes } = req.body;

  const client = await pool.connect();          // ১
  try {
    await client.query('BEGIN');                // ২

    const invoice = await client.query(
      'SELECT id, frozen_at FROM invoices WHERE id::TEXT = $1 FOR UPDATE', // ৩
      [invoice_id]
    );

    if (invoice.rowCount === 0) {
      await client.query('ROLLBACK');           // ৪
      return res.status(404).json({ message: 'No invoice with that id' });
    }
    if (invoice.rows[0].frozen_at) {
      await client.query('ROLLBACK');
      return res.status(409).json({ message: 'That invoice is already frozen...' });
    }

    const dispute = await client.query(
      `INSERT INTO disputes (invoice_id, filed_by, reason, notes)
      VALUES ($1, $2, $3, $4) RETURNING *`,
      [invoice_id, filed_by, reason, notes || null]
    );

    await client.query('UPDATE invoices SET frozen_at = NOW() WHERE id::TEXT = $1',
      [invoice_id]);          // ৫

    await logEvent(client, dispute.rows[0].id, 'Dispute filed', filed_by, reason);

    await client.query('COMMIT');                // ৬
    res.status(201).json(dispute.rows[0]);
  } catch (error) {
    await client.query('ROLLBACK');
    res.status(500).json({ message: 'Could not file the dispute' });
  } finally {
    client.release();          // ৭
  }
});
```

নং	কোড	কেন এভাবে
১	<code>pool.connect()</code>	<code>pool.query()</code> প্রতিবার আলাদা কানেকশন দিতে পারে। কিন্তু transaction একটাই কানেকশনে শুরু থেকে শেষ পর্যন্ত চলতে হয়
২	<code>BEGIN</code>	এখান থেকে <code>COMMIT</code> পর্যন্ত সব কাজ একসাথে হবে, নয়তো কিছুই হবে না
৩	<code>FOR UPDATE</code>	সবচেয়ে দামি লাইন। এই invoice-এর সারিটা তালাবদ্ধ হয়ে যায়। দুইজন একই সেকেন্ডে dispute চাপলে দ্বিতীয়জনকে অপেক্ষা করতে হয়, তারপর দেখে frozen হয়ে গেছে — তাই 409 পায়। এটা না থাকলে দুইজনই “frozen নয়” দেখে দুইটা dispute তৈরি করে ফেলত
৪	<code>ROLLBACK</code>	সব বাতিল। খেয়াল করুন — শুধু <code>catch</code> -এ নয়, মাঝপথে বেরিয়ে যাওয়ার সময়ও দেওয়া আছে, নইলে তালা খোলাই থাকত
৫	<code>frozen_at = NOW()</code>	এটাই freeze। এখান থেকে invoice-টা marketplace-এ আর দেখা যাবে না
৬	<code>COMMIT</code>	এখন সব পাকা হলো। এর আগে অন্য কেউ এই পরিবর্তন দেখতেই পেত না
৭	<code>finally { client.release() }</code>	কানেকশন pool-এ ফেরত দেওয়া। <code>finally</code> সফল-ব্যর্থ দুই ক্ষেত্রেই চলে। এটা ভুলে গেলে কিছুক্ষণ পর সার্ভার একদম আটকে যেত

Route-এর ক্রম — একটা লুকানো ফাঁদ

```
router.get('/invoice/:invoiceId', ...) // আগে
router.get('/:id', ...) // পরে
```

Express উপর থেকে নিচে মেলায়, প্রথম মিল পেলেই থেমে যায়। `/:id` আগে থাকলে সে “invoice” শব্দটাকেই একটা id ভেবে নিত, আর `/invoice/...` কখনো চলতই না।

৫.৫ apurba/InvoiceUpload.jsx — ফ্রন্টএন্ড

State (অ্যাপের স্মৃতি)

```
const [preview, setPreview] = useState(''); // যে ছবিটা দেখাচ্ছে
const [file, setFile] = useState(null); // আসল ফাইল
const [progress, setProgress] = useState(0); // OCR কত শতাংশ
const [reading, setReading] = useState(false); // পড়া চলছে কিনা
const [form, setForm] = useState({ buyer_name:'', invoice_number:'', ... });
```

`useState` হলো React-এর স্মৃতি। [মান, বদলানোর ফাংশন] — `setX()` ডাকলে React পেজটা আবার আঁকে।

PDF না ছবি — সিদ্ধান্ত

```
if (file.type === 'application/pdf') {
  text = await readPdf(file);
} else {
  setPreview(URL.createObjectURL(file));
  text = await runOcr(file);
}
```

পুরো ফিচারে এই একটাই দুইভাগে ভাগ। এরপর সব কোড একটা মাত্র *text* নিয়ে কাজ করে।

PDF-এর ভেতরের লেখা সরাসরি পড়া

```
const content = await page.getTextContent();
const layerText = joinTextItems(content.items);

if (layerText.trim().length > 40) {
  return layerText;      // OCR লাগেনি, একদম নিখুঁত
}
return runOcr(canvas);   // এটা স্ক্যান করা কাগজ, OCR চালাও
```

এটাই ফিচারের সবচেয়ে বুদ্ধিমান অংশ

Accounting software থেকে বানানো PDF-এর ভেতরে লেখাটা **আগে থেকেই থাকে**। তার ছবি তুলে OCR করলে নিখুঁত তথ্য ফেলে দিয়ে অনুমান করা হতো। ৪০ অক্ষরের শর্তটা আসল PDF আর স্ক্যান করা কাগজকে আলাদা করে — স্ক্যানে লেখা প্রায় শূন্য থাকে।

OCR চালানো

```
const result = await Tesseract.recognize(image, 'eng', {
  logger: (info) => {
    if (info.status === 'recognizing text') {
      setProgress(Math.round(info.progress * 100));
    }
  },
});
return result.data.text;
```

- **Tesseract.js** ব্রাউজারেই চলে — invoice-এর ছবি কোথাও পাঠানো হয় না, তাই গোপনীয়তা রক্ষা হয়।
- `logger` বারবার ডাকা হয়, `progress` ০ থেকে ১ — একে ১০০ দিয়ে গুণ করে progress bar দেখানো হয়।

চারটি তথ্য বের করা

```
const INVOICE_NUMBER_PATTERNS = [
  /invoice\s*(?:no\.?|number|#)\s*[:\-\]?\s*([A-Za-z0-9][A-Za-z0-9\/-]*...)/i,
  /\b(INV[-\s]?[A-Za-z0-9-]{3,})\b/i,
];

function findFirst(text, patterns) {
  for (const pattern of patterns) {
    const match = text.match(pattern);
    if (match) return match[1].trim();
  }
  return '';
}
```

- প্রত্যেক তথ্যের জন্য **একাধিক pattern**, কারণ ভিন্ন কোম্পানি ভিন্নভাবে লেখে — “Invoice No”, “Invoice #”, বা শুধু “INV-2026-1042”।
- `match[1]` নেওয়া হয়, `match[0]` নয় — কারণ ০ নম্বরে পুরো মিলটা থাকে (লেবেল সহ), ১ নম্বরে শুধু মানটা।
- `/i` মানে ছোট-বড় হাতের অক্ষরে পার্থক্য নেই।

তারিখের বড় সমস্যা

```
const numeric = value.match(/^(\d{1,2})[\/\-\s](\d{1,2})[\/\-\s](\d{2,4})$/);
if (numeric) {
  const day = numeric[1].padStart(2, '0'); // দিন আগে
  const month = numeric[2].padStart(2, '0');
  ...
}
```

এই বাগটা ভাইভাতে বলার মতো

JavaScript-এর `Date.parse("05/09/2026")` আমেরিকান নিয়মে পড়ে — **৯ মে**। বাংলাদেশে এর মানে **৫ সেপ্টেম্বর**। টাকার কাগজে চার মাসের ভুল, অথচ চোখে ধরা পড়ে না। তাই শুধু সংখ্যার তারিখ নিজে হাতে ভাগ করে দিন-আগে হিসেবে পড়া হয়। মাসের নাম লেখা থাকলে (“10 Nov 2026”) কোনো দ্বিধা নেই, তখন built-in parser নিরাপদ।

৬. API — ভেতরের ও বাইরের

৬.১ Internal API (আমার নিজের বানানো)

ফিচার ১ — `server/routes/invoiceRoutes.js`

#	Method	Endpoint	কাজ
০	POST	/api/invoices/upload-file	ফাইল Cloudinary-তে জমা
১	POST	/api/invoices/check-duplicate	invoice নম্বর আগে আছে কিনা
২	POST	/api/invoices	নতুন invoice সেভ
৩	GET	/api/invoices	সব invoice-এর তালিকা
৪	GET	/api/invoices/:id	একটা invoice
৫	PATCH	/api/invoices/:id	OCR-এর ভুল সংশোধন
৬	DELETE	/api/invoices/:id	ভুলে আপলোড করা invoice মুছে ফেলা

ফিচার ২ — `server/routes/disputeRoutes.js`

#	Method	Endpoint	কাজ
৭	POST	/api/disputes	Dispute করা + invoice freeze
৮	GET	/api/disputes?status=Open	Admin-এর তালিকা
৯	GET	/api/disputes/invoice/:invoiceId	এই invoice কি frozen?
১০	POST	/api/disputes/:id/evidence	প্রমাণের কাগজ যোগ
১১	GET	/api/disputes/:id	একটা dispute + প্রমাণ + ইতিহাস
১২	DELETE	/api/disputes/:id/evidence/:evidenceId	ভুল কাগজ সরানো
১৩	PATCH	/api/disputes/:id/resolve	Admin-এর সিদ্ধান্ত

৬.২ External API / Service

নাম	কী কাজে	কোন ফাইলে
Cloudinary	Invoice ও প্রমাণের ডকুমেন্ট জমা রাখা	server/routes/invoiceRoutes.js
Tesseract.js	ছবি থেকে লেখা পড়া (OCR)	client/src/apurba/InvoiceUpload.jsx
pdf.js	PDF-এর ভেতরের লেখা পড়া	client/src/apurba/InvoiceUpload.jsx
Supabase PostgreSQL	সব ডেটা জমা	server/db.js

৬.৩ Cloudinary Integration — লাইন বাই লাইন

ধাপ ১ — ব্রাউজার ফাইলটাকে টেক্সটে রূপান্তর করে

```
function readAsDataUri(theFile) {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = () => resolve(reader.result);
    reader.onerror = () => reject(new Error('Could not read the file'));
    reader.readAsDataURL(theFile);
  });
}
```

- FileReader ফাইল পড়ে ... এরকম একটা লম্বা স্ট্রিং বানায়।
- এটা callback-ভিত্তিক, তাই Promise দিয়ে মুড়ে দেওয়া হয়েছে যাতে await ব্যবহার করা যায় এবং কোড পড়তে সহজ হয়।

ধাপ ২ — নিজের সার্ভারে পাঠানো

```
const stored = await fetch(API_URL + '/api/invoices/upload-file', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    file: await readAsDataUri(file),
    file_name: fileName,
  }),
});
const storedData = await stored.json();

if (!stored.ok) { ...ভুল দেখাও, থেমে যাও... }
documentUrl = storedData.file_url;
```

অংশ	ব্যাখ্যা
<code>method: 'POST'</code>	নতুন কিছু তৈরি করছি, তাই POST
<code>headers</code>	সার্ভারকে বলা হচ্ছে body-টা JSON — নইলে <code>express.json()</code> বুঝত না
<code>JSON.stringify</code>	JavaScript object-কে টেক্সটে রূপান্তর, কারণ HTTP-তে টেক্সটই যায়
<code>stored.ok</code>	status 200-299 হলে true

ধাপ ৩ — সার্ভার Cloudinary-তে পাঠায়

```
const uploaded = await cloudinary.uploader.upload(file, {  
  folder: 'clarity/invoices',  
  resource_type: 'auto',  
});  
res.status(201).json({ file_url: uploaded.secure_url });
```

ভাইভার জন্য মূল কথা

secret কখনো ব্রাউজারে যায় না। ব্রাউজার শুধু আমার নিজের সার্ভারের সাথে কথা বলে; Cloudinary-র key ও secret `.env` ফাইলে সার্ভারে থাকে। কেউ browser-এর developer tools খুলেও সেটা দেখতে পারবে না।

আরেকটা সিদ্ধান্ত: **আগে ডকুমেন্ট জমা, পরে invoice সেভ**। উল্টো করলে এমন invoice তৈরি হতে পারত যার কাগজটাই নেই।

৭. ভাইভা প্রস্তুতি — ১০টি সম্ভাব্য প্রশ্ন

প্রশ্ন ১

আপনার প্রজেক্টে কী কী ফিচার আপনি বানিয়েছেন?

উত্তর: আমি দুটি ফিচার করেছি। এক, **Invoice Upload with OCR** — সাপ্লায়ার invoice-এর ছবি বা PDF দিলে সিস্টেম নিজে পড়ে ক্রেতার নাম, invoice নম্বর, টাকা ও তারিখ বের করে ফর্ম পূরণ করে দেয়, একই invoice দুইবার জমা পড়া ঠেকায়, আর কাগজটা Cloudinary-তে রাখে। দুই, **Dispute Filing & Invoice Freeze** — ক্রেতা কোনো invoice নিয়ে আপত্তি করলে সেটা সঙ্গে সঙ্গে freeze হয়ে যায় যাতে কেউ ওতে টাকা না লাগায়, প্রমাণ যোগ করা যায়, আর admin release বা void করতে পারে।

প্রশ্ন ২

SQL Injection কী? আপনি কীভাবে ঠেকিয়েছেন?

উত্তর: SQL Injection হলো ইনপুটের ভেতরে SQL কমান্ড লুকিয়ে পাঠিয়ে ডেটাবেসের ক্ষতি করা। আমি কোনো জায়গাতেই স্ট্রিং জোড়া দিয়ে query বানাইনি — সব মান \$1 , \$2 placeholder দিয়ে আলাদাভাবে পাঠিয়েছি। PostgreSQL আগে query-টা বুঝে নেয়, তারপর মান বসায়, তাই মানটা কখনো কমান্ড হিসেবে চলতে পারে না। আমি পরীক্ষাও করেছি — ?supplierId=1;DROP TABLE invoices পাঠিয়েছিলাম, টেবিল অক্ষত ছিল।

প্রশ্ন ৩

Transaction কী? কোথায় ব্যবহার করেছেন এবং কেন?

উত্তর: Transaction মানে কয়েকটা কাজকে একটা অবিভাজ্য একক করা — হয় সবগুলো হবে, নয় একটাও হবে না। Dispute করার সময় দুটো কাজ হয়: dispute-এর রেকর্ড তৈরি, আর invoice freeze। যদি প্রথমটা হয়ে দ্বিতীয়টা ব্যর্থ হতো, তাহলে বিতর্কিত invoice-টা marketplace-এ পড়ে থাকত এবং কেউ ওতে টাকা লাগিয়ে দিত। তাই BEGIN থেকে COMMIT পর্যন্ত একসাথে, আর যেকোনো সমস্যায় ROLLBACK।

প্রশ্ন ৪

দুইজন একই মুহূর্তে একই invoice dispute করলে কী হবে?

উত্তর: প্রথম অনুরোধ `SELECT ... FOR UPDATE` দিয়ে সারিটা তালাবদ্ধ করে ফেলে। দ্বিতীয়জনকে অপেক্ষা করতে হয়; তাল খোলার পর সে দেখে `frozen_at` বসে গেছে, তাই 409 Conflict পায়। এছাড়া ডেটাবেসেও একটা *partial unique index* দিয়েছি — একটা invoice-এর বিপরীতে একসাথে একটাই Open dispute থাকতে পারে। কোড যাই করুক, ডেটাবেস নিজেই দ্বিতীয়টা আটকে দেবে।

প্রশ্ন ৫

OCR ভুল পড়লে কী হবে?

উত্তর: প্রতিটি ঘর সাধারণ editable input — সাপ্লায়ার নিজের চোখে দেখে ঠিক করে নেয়। এটা কোনো দুর্বলতা নয়, স্পেসিফিকেশনেই বলা আছে সাপ্লায়ার confirm করার আগে যাচাই করবে। কোন কোন ঘর সিস্টেম নিজে পূরণ করেছে সেটাও দেখানো হয়, আর OCR আসলে কী পড়েছে সেই কাঁচা লেখাটাও খুলে দেখা যায়। আমার `5-photo-skewed.png` নমুনাটা ইচ্ছে করেই বাঁকা ও ব্যাপসা — ওটা দিয়ে দেখানো যায় ভুল হলে কী হয়।

প্রশ্ন ৬

PDF আর ছবির জন্য আলাদা ব্যবস্থা কেন?

উত্তর: Accounting software থেকে তৈরি PDF-এর ভেতরে লেখাটা আগে থেকেই ডিজিটাল আকারে থাকে। ওটার ছবি বানিয়ে OCR করলে নিখুঁত তথ্য ফেলে দিয়ে আন্দাজ করা হতো। তাই আগে PDF-এর text layer পড়ি; যদি ৪০ অক্ষরের কম পাই তার মানে ওটা আসলে স্ক্যান করা কাগজ, তখন OCR চালাই। একই ফিচারে দুটো পথ, কিন্তু ফলাফল একটাই — সাদা লেখা।

প্রশ্ন ৭

Cloudinary-র secret আপনি কোথায় রেখেছেন?

উত্তর: `server/.env` ফাইলে, আর সেই ফাইল `.gitignore`-এ আছে তাই GitHub-এ যায় না। ব্রাউজার কখনো Cloudinary-র সাথে সরাসরি কথা বলে না — সে শুধু আমার সার্ভারে ফাইলটা পাঠায়, সার্ভার Cloudinary-তে আপলোড করে লিংকটা ফেরত দেয়। ফলে key বা secret কোনোদিন ব্রাউজারে পৌঁছায় না।

প্রশ্ন ৮

Freeze করতে status কলাম ব্যবহার না করে আলাদা কলাম কেন?

উত্তর: আমাদের requirement অনুযায়ী invoice-এর status কলামটা SL 2-এর **state machine** সার্ভিসের অধীনে — নিয়ম হলো ওই সার্ভিস ছাড়া কেউ ওটা লিখবে না। তাই আমি frozen_at নামে আলাদা একটা timestamp কলাম রেখেছি। এতে dispute invoice-টা freeze করতে পারে, অথচ দলের নিয়মও ভাঙে না।

প্রশ্ন ৯

409 status code কেন ব্যবহার করলেন, 400 নয় কেন?

উত্তর: 400 মানে অনুরোধটাই ভুল — ঘর খালি, টাকার জায়গায় অক্ষর। কিন্তু “এই invoice আগেই frozen” বা “এই dispute আগেই মীমাংসা হয়ে গেছে” — এখানে অনুরোধ একদম ঠিক আছে, শুধু ডেটার এখনকার অবস্থা কাজটা করতে দিচ্ছে না। এটাই **409 Conflict**। ক্লায়েন্ট 400 দেখে ইনপুট ঠিক করবে, আর 409 দেখে বুঝবে অন্য কিছু বদলাতে হবে।

প্রশ্ন ১০

আপনার সিস্টেমে কী কী দুর্বলতা আছে?

উত্তর: সবচেয়ে বড় দুর্বলতা — এখনো **authentication নেই**, তাই যে কেউ URL-এ supplier id বদলে অন্যের ডেটা দেখতে পারে। তবে registration, login ও KYB আমাদের requirement-এ *common workflow* হিসেবে আছে, সেটা পুরো দলের কাজ, আমার এই দুই ফিচারের অংশ নয়। ঠিক করার উপায়ও জানি — supplier id ব্রাউজারের পাঠানো query থেকে না নিয়ে যাচাই করা session token থেকে নিতে হবে। এছাড়া invoice নম্বরে এখনো database-level unique constraint দিইনি (কোডে চেক আছে), আর CSV export-এ কমা সামলানো আছে কিন্তু উদ্ভৃতি চিহ্ন নেই।

৮. দ্রুত রিভিশন — ভাইভার আগে একবার চোখ বুলিয়ে নিন

শব্দ	এক লাইনে মানে
API	দুই প্রোগ্রামের কথা বলার নিয়ম। আমার ক্ষেত্রে — ব্রাউজার সার্ভারকে যে URL-এ ডাকে
Endpoint	একটা নির্দিষ্ট URL + method, যেমন <code>POST /api/disputes</code>
Middleware	প্রত্যেক অনুরোধের আগে যে ফাংশন চলে, যেমন <code>express.json()</code>
Router	একই ফিচারের কয়েকটা endpoint একসাথে রাখা ছোট অ্যাপ
Pool	খুলে রাখা কয়েকটা ডেটাবেস কানেকশন
Transaction	হয় সব কাজ হবে, নয় একটাও হবে না
FOR UPDATE	সারিটা তালাবদ্ধ করা, যাতে দুইজন একসাথে বদলাতে না পারে
useState	React-এর স্মৃতি; বদলালে পেজ আবার আঁকা হয়
useEffect	পেজ আসার পর যে কাজ চলে, যেমন ডেটা আনা
Controlled input	যে ঘরের মান React state থেকে আসে
OCR	ছবির ভেতরের লেখা চিনে টেক্সটে রূপান্তর
data URI	ফাইলকে <code>data:image/png;base64,...</code> স্ট্রিং বানানো
NUMERIC(12,2)	নিখুঁত দশমিক — টাকার জন্য; float-এ ভগ্নাংশে ভুল হয়
LEFT JOIN	বাঁ পাশের সব সারি রাখে, মিল না পেলে ফাঁকা দেখায়

শেষ পরামর্শ

ভাইভাতে দুটো জিনিস দেখালেই সবচেয়ে ভালো ফল পাবেন — (১) একই **invoice** দুইবার **submit** করে duplicate ধরা পড়া, আর (২) একই **invoice** দুইবার **dispute** করে “already frozen” বার্তা। দুটোই এক ক্লিকের কাজ, আর দুটোই দেখায় যে সিস্টেমটা শুধু সুন্দর নয়, নিয়মও প্রয়োগ করে।